KV Forge: Mastering LLM Inference, from KV Cache to Production — Advanced

Portfolio projects · lessons 26–40

Each takes 1–3 days and ends with something to show an employer.

Project 1: Brightside replica gateway: measure routing before you trust it

Goal: Build the three-replica Brightside serving cluster, run the same 1,000 policy-heavy requests under three routing rules, and show with numbers which rule keeps p95 TTFT low, spreads load evenly and survives a replica failure. This is the running example at production size.

Uses lessons: 07, 26, 27, 29, 30, 33

1. Start the lesson 30 Compose cluster with three mock vLLM replicas (QUEUE_LIMIT=8), one gateway and Prometheus, and pin the image tags the lesson lists; confirm each replica's /metrics page shows vllm:-prefixed names before you run anything.

2. Generate a seeded workload of 1,000 requests with `random.seed(30)`: each request is a 1,500-word policy prompt from one of three policy families plus a 20 to 60 word question; 80% ask for a 32-token answer and 20% for 320. Save it as data/workload.jsonl and write the seed and the file's SHA-256 in the README.

3. Replay the same workload with 24 requests in flight under round-robin, token-budget and prefix-aware routing, three times each (nine runs). For each run record p50 and p95 TTFT, the requests per replica (max divided by min), and the prefix hit rate from vllm:prefix_cache_hits_total over vllm:prefix_cache_queries_total.

4. Pick the rule with the lowest mean p95 TTFT among the rules whose busiest replica gets no more than 1.5 times the least busy one's requests, and write one sentence in the README on why a rule that wins on cache hits can lose on balance.

5. Add admission control: cap each replica's queue at 8 requests, set proxy_next_upstream_tries 2 in the gateway so a 503 is retried once on another replica, then raise the load to 48 requests in flight and count retries and vllm:admission_rejections_total for the 1,000 requests.

6. Stop one replica at request 500 of a 1,000-request run and record how many requests failed, how many were retried, and p95 TTFT in the 60 seconds before and after the stop; repeat the stop three times and report the mean.

7. Write the README with one table (three rules by mean and range over three runs for p95 TTFT, balance ratio and hit rate), the commands used, and a line stating these are CPU mock-engine results that test routing logic, not GPU speed.

Deliverable: Git repo with the Compose file, the gateway config, the workload generator, the replay script and a README with the nine-run table, the failure-drill numbers and the chosen rule with its reason.

Stretch: Vary the prefix block size over 8, 16 and 32 tokens and plot hit rate against memory used per replica. · Add a fourth replica and report scaling efficiency: p95 TTFT at 24 and 32 requests in flight, per replica.

Project 2: Overnight claims triage for an insurer: capacity and cost before the purchase order

Goal: For a different, realistic domain (an insurer's nightly claims-triage batch), size the GPUs from a Little's-law model, measure p95 latency under open-loop load, and decide build versus buy from cost per million tokens at three utilisation levels. Shows the cost reasoning a staff engineer must defend to finance.

Uses lessons: 07, 26, 27, 28

1. Write the workload spec in the README: 120,000 claims a night, an 8-hour window, and a peak hour that carries 25% of arrivals (30,000 claims, 8.3 per second). Use claim text from a public, openly licensed dataset of at least 500 documents, recording its name, URL and licence in data/SOURCES.md; if nothing fits, generate lengths from a seeded distribution and label them as made up.

2. Size the fleet with Little's law: concurrency = arrival rate x time in system. At 8.3 claims per second and a measured 12 seconds per claim, that is about 100 requests in flight; add 30% headroom and convert to GPUs using the slots per GPU you measured in step 3.

3. Run an open-loop load test with Poisson arrivals at 50%, 80% and 100% of the measured capacity, 10 minutes per level, with input and output lengths varied by plus or minus 50%; keep at least 500 requests per level and report p95 TTFT and goodput under a 5-second TTFT target.

4. Compute cost per million output tokens at 20%, 50% and 80% utilisation as hourly GPU cost divided by tokens per hour; enter the GPU hourly price from a dated provider price page and mark it as unverified if you could not open the page.

5. Measure the cache lever: run the 500-request set with the shared 1,500-token claims-policy prompt with prefix caching on and off, read the hit rate from vllm:prefix_cache_hits_total and vllm:prefix_cache_queries_total, and recompute cost per million input tokens at that hit rate.

6. Compare build with buy: take a hosted API price per million tokens from its pricing page (with the date), and find the monthly claim volume where a reserved GPU fleet becomes cheaper, both as claims per month and as a utilisation percentage.

7. Write the decision memo in the README: one table at 20%, 50% and 80% utilisation, the breakeven volume, the recommendation, and the source of every number, with lab-measured numbers kept apart from recorded or quoted ones.

Deliverable: Git repo with the workload generator, the load-test scripts, the cost calculator with its inputs in one config file, the recorded results, and a README with the Little's-law sizing, the three-level cost table, the cache comparison, the breakeven and the recommendation.

Stretch: Add a spot-instance scenario with a 30% preemption rate per hour and measure how many claims must be retried and what that does to cost per claim. · Split the batch into a short-claim pool and a long-claim pool and show whether two pools beat one pool on p95 TTFT at the same cost.

Project 3: Brightside two-region failover drill with an error budget

Goal: Run the Brightside serving service across two regions in a local Compose lab, state an SLO and an error budget in numbers, then drill a full regional loss under load and prove the recovery time and budget spent. This is the production-operations skill that interviewers probe for SRE and platform roles.

Uses lessons: 26, 31, 33, 35, 37, 38, 39

1. Build the two-region Compose lab from lesson 38: two regional gateways, two mock replicas in each region and a global nginx on the port from `progress.py --ports 38`; confirm that healthy traffic reaches region a only.

2. Write the SLO in the README: 99.5% of requests succeed and p95 TTFT is under 2 seconds over a rolling 4-week window. At 10 requests per second that is 24,192,000 requests a window and a budget of 120,960 failed requests.

3. Run Locust at 20 requests per second for 15 minutes, stop region a (both replicas and its gateway) at minute 5 and start it again at minute 10; repeat the drill three times and record failed requests, seconds until the first successful response from region b, and p95 TTFT during and after the outage.

4. Tune failover so one lost region costs less than 1% of a drill's requests: test max_fails 1 and 2 and fail_timeout 5s and 10s, pick the pair with the fewest failures, and state the values and the failure counts for each pair.

5. Measure the weight-restore path: write a 2 GB file to stand in for weights, copy it to a second directory standing in for the second region's bucket, time the copy, and extrapolate to about 140 GB for Llama 3.1 70B in FP16 at the measured rate; then state the replication lag you would accept against the 12-hour default in the RESEARCH notes.

6. Write the incident runbook in the README: who decides to fail over, the rule that freezes rollouts while a region is down, the measured recovery time per drill, and the budget spent per drill as a percentage of the 120,960 allowed failures scaled to the drill's request count.

Deliverable: Git repo with the Compose lab, the nginx configs, the Locust file, the drill scripts, the restore-timing script and a README with the SLO, the three-drill table, the chosen failover values, the restore extrapolation and the runbook.

Stretch: Write two Prometheus burn-rate alerts for the SLO (1-hour and 5-minute windows), test them with promtool against the drill's recorded metrics, and show when they fire. · Run the drill with region b at 80% of its normal capacity and add load shedding; report how many requests are rejected and whether p95 TTFT stays under 2 seconds.

Project 4: Card-dispute assistant for a bank: tenancy, redaction and a design review

Goal: For a regulated bank's card-dispute assistant, prove that one tenant's prompts cannot leak into another's cache, that card numbers and contact details stay out of logs, and that the endpoints an API key does not protect are closed. End with a 6-to-10-page design that a principal engineer would sign off.

Uses lessons: 07, 08, 09, 36, 38, 39, 40

1. List every endpoint of the lab server (the chat and completion routes, /metrics, /health, the docs page and any tokenize or admin route) in a table; send each one a request with and without the API key and record the status codes; block or firewall each route that answers without a key.

2. Write a log redaction filter that removes card-like numbers of 13 to 19 digits that pass the Luhn check, IBAN-like strings, email addresses and phone numbers; test it on a fixed set of 200 made-up lines (50 with a card number, 50 with an email, 100 clean) and report recall and false positives. Use only generated test numbers, never real card data.

3. Set a cache_salt per business unit (cards, disputes, branch support) from .env, run the same 1,500-token prompt from unit A and then from unit B on the vLLM CPU image pinned in lesson 36, and show the prefix-cache hits for unit B with salts off (a leak) and with salts on (zero hits).

4. Write an audit line per request as JSON: time, business unit, a hash of the key id, model version, input and output token counts, latency and a request id, with no prompt or answer text; run 300 requests and grep the logs for 20 planted test card numbers and report the match count.

5. Size the platform for 300 requests per second at peak across two regions and three business units, using the scale table in lesson 38 and the tokens per second you measured in lesson 27 or recorded for the same model; state every assumption, including the output length and the cache hit rate.

6. Write the design document (6 to 10 pages, as a README plus a docs folder) covering the threat list, tenancy rules, data residency (each unit's card data stays in its home region), the redaction and audit design, the capacity model and the failure modes from lesson 33, with a section listing what you cut and why.

Deliverable: Git repo with the endpoint test script, the redaction filter and its test set, the cache-isolation test, the audit logger and its grep check, the capacity model, and a design document of 6 to 10 pages with every measured number labelled.

Stretch: Rotate one business unit's cache_salt in the middle of a test run and show that the old cached prefix is unreachable while the other units keep their hit rates. · Add a per-unit rate limit at the gateway and show that a noisy unit sending 3 times its quota cannot push another unit's p95 TTFT above 2 seconds.